Reescrita de Consultas SQL Baseada em Modelos de Linguagem de Grande Escala (LLM-based SQL Query Rewriting)

Luiz Eduardo Treméa[1], **Fahad Kalil**[1]

[1]Ciência da Computação – Atitus Educação
Passo Fundo – RS – Brasil

1123471@atitus.edu.br, fahad.kalil@atitus.edu.br

Abstract. *Escreva seu resumo em língua estrangeira (inglês)...*

Resumo. *Resumo do trabalho (português)...*

1. Introdução

Cenários, motivação, justificativa, problema de pesquisa, objetivo geral e específicos (taxonomia de Bloom);

2. Referencial Teórico

2.1. Evolução da Otimização de Consultas e Uso de Inteligência Artificial

A otimização de consultas em sistemas de gerenciamento de bancos de dados tem sido historicamente sustentada por três componentes centrais: a estimação de cardinalidade, o modelo de custo e a enumeração de planos. Nesse contexto, a qualidade do plano gerado depende diretamente da precisão das estimativas de volume de dados intermediários e da adequação da função de custo associada aos recursos computacionais.

Apesar de amplamente adotadas, essas abordagens apresentam limitações significativas em cenários contemporâneos, especialmente quando os dados apresentam correlações complexas entre atributos ou quando há variações frequentes na carga de trabalho. Nesses casos, erros de estimação podem comprometer a seleção do plano físico, resultando em degradação de desempenho (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021).

Nesse contexto, surge o paradigma de *Learned Query Optimization* (LQO), no qual técnicas de aprendizado de máquina são incorporadas ao processo de otimização. O sistema *Neo* propõe um otimizador baseado em redes neurais profundas capaz de aprender padrões de execução a partir de consultas reais, adaptando-se dinamicamente ao comportamento dos dados (Marcus; Negi; Mao; Zhang *et al.*, 2019). De forma complementar, o sistema *Bao* introduz um mecanismo de otimização híbrida que utiliza aprendizado por reforço para fornecer sugestões de execução por consulta, melhorando o desempenho em cenários reais (Marcus; Negi; Mao; Tatbul *et al.*, 2021).

2.2. Reescrita de Consultas SQL com Modelos de Linguagem

A reescrita de consultas SQL tem como objetivo transformar uma consulta em outra semanticamente equivalente, porém mais eficiente do ponto de vista computacional. Diferentemente de abordagens de *Text-to-SQL*, o foco da reescrita reside em modificar consultas já estruturadas, mantendo a equivalência lógica dos resultados.

O método *LLM-R2* utiliza modelos de linguagem para sugerir regras de reescrita, combinando aprendizado contrastivo para aprimorar representações de consultas (Li *et al.*, 2024). De maneira semelhante, o sistema *R-Bot* emprega uma abordagem baseada em recuperação de evidências estruturais e semânticas, com o objetivo de orientar o modelo e reduzir erros durante o processo de reescrita (Sun *et al.*, 2025).

Além disso, o sistema *QUITE* utiliza uma arquitetura baseada em múltiplos agentes que interagem por meio de um fluxo controlado, explorando estratégias de reescrita com realimentação do sistema de banco de dados (Song *et al.*, 2025). Já o sistema *LITHE* propõe mecanismos adicionais para garantir a correção semântica das consultas geradas, incorporando validações baseadas em análise lógica e estatística (Dharwada *et al.*, 2026).

2.3. Estimativa de Cardinalidade e Modelos Aprendidos

A estimação de cardinalidade é amplamente reconhecida como um dos principais fatores que influenciam a qualidade do plano de execução em sistemas de banco de dados, pois determina a previsão do volume de dados manipulados em cada etapa da execução da consulta.

O modelo *NeuroCard* propõe um estimador baseado em redes neurais autorregressivas capaz de capturar correlações complexas entre múltiplas tabelas sem assumir independência entre atributos (Yang *et al.*, 2021). Por outro lado, o método *FLAT* introduz um modelo gráfico que combina fatoração independente e condicional, buscando equilibrar precisão e eficiência computacional (Zhu *et al.*, 2021).

Essas abordagens evidenciam uma evolução significativa em relação aos métodos tradicionais, permitindo estimativas mais precisas e impactando diretamente o desempenho das consultas executadas.

2.4. Arquiteturas Baseadas em Recuperação e Agentes

Abordagens recentes têm explorado a integração de modelos de linguagem com mecanismos de recuperação de conhecimento, formando pipelines baseados em *Retrieval-Augmented Generation* (RAG). Nesses sistemas, consultas são convertidas em representações vetoriais para recuperar evidências relevantes que orientam o processo de geração.

O sistema *R-Bot* exemplifica esse modelo ao combinar recuperação híbrida com geração guidada por evidências (Sun *et al.*, 2025). De forma complementar, o *QUITE* utiliza múltiplos agentes responsáveis por planejamento, validação e execução de reescritas, promovendo um processo iterativo de aprimoramento (Song *et al.*, 2025).

Essas arquiteturas demonstram que a utilização de conhecimento externo e mecanismos de validação iterativa contribuem significativamente para reduzir erros e aumentar a robustez das soluções baseadas em modelos de linguagem (Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

2.5. Síntese Crítica da Literatura

A literatura recente evidencia duas direções principais na evolução da otimização de consultas. A primeira envolve a incorporação de aprendizado de máquina diretamente no núcleo do otimizador, com exemplos como *Neo*, *Bao*, *NeuroCard* e *FLAT* (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021; Yang *et al.*, 2021; Zhu *et al.*, 2021).

A segunda direção explora a reescrita externa de consultas por meio de modelos de linguagem, como observado em *LLM-R2*, *R-Bot*, *QUITE* e *LITHE* (Li *et al.*, 2024; Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

De forma geral, os principais desafios atuais envolvem a garantia de correção semântica, a redução da latência dos modelos e a integração eficiente com sistemas existentes. Nesse cenário, a combinação entre aprendizado de máquina, reescrita de consultas e mecanismos de recuperação de conhecimento representa um caminho promissor para avanços na área.

3. Trabalhos Relacionados

Nesta seção, são analisados os principais estudos que utilizam Inteligência Artificial, especificamente LLMs e modelos aprendidos, para a reescrita e otimização de consultas SQL.

- **O uso da Inteligência Artificial na otimização de consultas em bancos de dados (Amaral, Andrade e Campos, 2025)**
  Os autores investigaram a viabilidade do uso do ChatGPT para a reescrita semântica de consultas em ambientes PostgreSQL. O estudo utilizou uma metodologia experimental com scripts Python para automatizar a execução e coleta de métricas, focando em transformações como o uso de *Common Table Expressions* (CTEs), pré-agregações e redução de subconsultas. Os resultados demonstraram que a intervenção da IA pode reduzir o tempo de execução em até 90% em cenários complexos com múltiplos *joins*, promovendo uma racionalização no uso de índices e tabelas.
- **LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System (Li et al., 2024)**
  Este trabalho propõe o sistema LLM-R2, que utiliza LLMs para sugerir sequências de regras de reescrita para plataformas como o Apache Calcite. O diferencial técnico reside no uso de um modelo contrastivo treinado via currículo para selecionar demonstrações (*few-shot*) estruturalmente similares à consulta de entrada. Ao delegar a execução final das regras a um motor de banco de dados, o sistema garante executabilidade e equivalência semântica, superando abordagens de geração livre suscetíveis a alucinações.
- **R-Bot: An LLM-based Query Rewrite System (Sun et al., 2025)**
  O estudo introduz o R-Bot, um sistema que busca mitigar alucinações por meio de uma arquitetura baseada em *Retrieval-Augmented Generation* (RAG). O sistema utiliza recuperação híbrida, combinando características estruturais e semânticas, para buscar "receitas de reescrita" e evidências provenientes de fóruns técnicos e manuais de SGBDs. O processo é refinado por um mecanismo de reescrita passo a passo com autorreflexão, permitindo que o modelo compreenda as inter-relações entre regras antes da aplicação final.
- **LITHE: A Query Rewrite Advisor using LLMs (Dharwada et al., 2025)**
  Os autores propõem o LITHE, sistema que realiza transformações diretamente no espaço de consulta (*SQL-to-SQL*) em vez do espaço de plano físico. A abordagem utiliza regras sensíveis ao banco de dados, fundamentadas em seletividade e metadados, combinadas com uma busca em árvore de Monte Carlo (*Monte Carlo Tree*

Search – MCTS) guiada pelas probabilidades de *tokens* do LLM. Essa estratégia permite explorar múltiplos caminhos de reescrita quando a confiança do modelo é baixa, alcançando acelerações expressivas em comparação aos otimizadores nativos.

- **QUITE: A Query Rewrite System Beyond Rules with LLM Agents (Song et al., 2025)**
  Este trabalho apresenta o QUITE, um *framework* baseado em múltiplos agentes controlados por uma Máquina de Estados Finita (*Finite State Machine – FSM*). O sistema utiliza feedback em tempo real do banco de dados, obtido via comandos `EXPLAIN`, além de técnicas de injeção de *hints* para garantir que o plano otimizado seja seguido pelo motor de execução. Diferentemente de métodos puramente baseados em regras, o QUITE permite a descoberta de estratégias de otimização que não podem ser expressas por transformações fixas.

Diferente das abordagens de otimização interna, como Neo e Bao, que buscam substituir ou modificar componentes do otimizador do SGBD, e dos sistemas de reescrita baseados exclusivamente em regras ou focados apenas na redução isolada de latência, este trabalho propõe uma abordagem de reescrita de consultas SQL do tipo *SQL-to-SQL* baseada em Modelos de Linguagem de Grande Escala (LLMs), apoiada por mecanismos de validação e análise conduzidos pelo próprio sistema gerenciador de banco de dados.

O diferencial da proposta reside na busca por um equilíbrio entre desempenho e preservação semântica. Enquanto abordagens de geração livre oferecem maior flexibilidade de transformação, frequentemente sofrem com problemas de *semantic drift*, alucinações e regressões de desempenho, sistemas rigidamente baseados em regras limitam o espaço de otimização a transformações previamente codificadas. Neste contexto, o presente trabalho propõe uma camada externa e não invasiva de otimização, na qual o LLM atua como agente de reescrita semântica, enquanto o SGBD exerce o papel de mecanismo de validação e evidência técnica.

A abordagem proposta estrutura-se em múltiplos estágios de verificação. Inicialmente, a consulta reescrita é submetida à validação sintática por meio do *parser* do banco de dados. Em seguida, comandos como `EXPLAIN` e `EXPLAIN ANALYZE` são utilizados para analisar o plano de execução e comparar o custo estimado entre a consulta original e a versão otimizada. Por fim, a equivalência da transformação é avaliada por meio da comparação dos resultados produzidos pelas consultas, reduzindo o risco de alterações indevidas na semântica original.

Dessa forma, o sistema proposto estabelece um ciclo em que o LLM propõe transformações, o otimizador do SGBD fornece evidências de desempenho e os mecanismos de validação asseguram a correção da reescrita. Como resultado, busca-se desenvolver uma solução de otimização prática, explicável e compatível com diferentes SGBDs relacionais, preservando a confiabilidade necessária para aplicações corporativas e ambientes reais de produção.

4. Materiais e Métodos

Este trabalho caracteriza-se como uma pesquisa aplicada, de abordagem quantitativa e caráter experimental. A natureza aplicada decorre da busca por uma solução para um

problema real relacionado à otimização de consultas SQL em sistemas gerenciadores de bancos de dados. A abordagem quantitativa é adotada por utilizar métricas objetivas de desempenho para avaliar os resultados obtidos, enquanto o caráter experimental está associado à implementação de um protótipo funcional e à realização de testes controlados em ambiente computacional.

A proposta consiste no desenvolvimento de um mecanismo de reescrita de consultas SQL baseado em Modelos de Linguagem de Grande Escala (Large Language Models – LLMs), complementado por etapas de validação sintática, validação semântica e análise de desempenho baseada no feedback do próprio otimizador do banco de dados. O objetivo é verificar se consultas reescritas automaticamente podem apresentar melhor desempenho sem comprometer a corretude dos resultados produzidos.

4.1. Arquitetura da Solução Proposta

A arquitetura proposta é composta por uma camada intermediária responsável por receber consultas SQL, encaminhá-las para um modelo de linguagem e avaliar a viabilidade das transformações sugeridas.

Inicialmente, uma consulta SQL original é submetida ao sistema. Em seguida, a consulta é enviada ao LLM, que recebe instruções específicas para produzir uma versão semanticamente equivalente e potencialmente mais eficiente. Após a geração da nova consulta, são executadas etapas de validação destinadas a verificar sua corretude e seu impacto sobre o desempenho.

A solução foi concebida para atuar de forma não invasiva, sem necessidade de alterações no núcleo do Sistema Gerenciador de Banco de Dados (SGBD). Dessa forma, o mecanismo pode ser utilizado como uma camada complementar aos otimizadores já existentes.

4.2. Processo de Reescrita de Consultas

O processo de reescrita tem início com a submissão de uma consulta SQL ao sistema. A consulta original é utilizada como entrada para o modelo de linguagem, que recebe um conjunto de instruções voltadas à otimização da estrutura lógica da consulta.

As transformações esperadas incluem a substituição de subconsultas por junções, utilização de Common Table Expressions (CTEs), simplificação de predicados, reorganização de filtros e eliminação de operações redundantes. Essas estratégias foram identificadas na literatura como mecanismos capazes de reduzir custos de execução e melhorar o aproveitamento dos recursos computacionais.

O modelo de linguagem não possui autonomia para substituir automaticamente a consulta original. Toda reescrita gerada deve passar pelas etapas de validação definidas neste trabalho antes de ser considerada uma alternativa válida.

4.3. Validação das Consultas Reescritas

Considerando os riscos de alucinações e desvios semânticos observados na literatura, foi definida uma estratégia de validação em múltiplos estágios.

A primeira etapa consiste na validação sintática, realizada pelo próprio parser do banco de dados. Consultas que apresentarem erros de compilação ou incompatibilidades estruturais são descartadas automaticamente.

Na sequência, é realizada a validação semântica. Nessa etapa, a consulta original e a consulta reescrita são executadas sobre o mesmo conjunto de dados e seus resultados são comparados. Caso sejam identificadas diferenças entre os conjuntos retornados, a reescrita é rejeitada.

Por fim, realiza-se a validação de desempenho utilizando informações fornecidas pelo próprio otimizador do banco de dados. São analisados os planos de execução gerados pelos comandos `EXPLAIN` e `EXPLAIN ANALYZE`, permitindo comparar custos estimados, operadores utilizados e tempos de execução observados.

Somente consultas que preservarem a equivalência semântica e apresentarem melhoria de desempenho serão consideradas otimizações válidas.

4.4. Ambiente Experimental

Os experimentos serão realizados utilizando o PostgreSQL como Sistema Gerenciador de Banco de Dados. A escolha ocorre em função de sua ampla utilização em ambientes acadêmicos e corporativos, bem como pela disponibilidade de ferramentas nativas de análise de desempenho.

O protótipo será desenvolvido na linguagem Python, escolhida por sua facilidade de integração com bancos de dados relacionais e plataformas de Inteligência Artificial.

Entre as tecnologias utilizadas destacam-se:

- PostgreSQL;
- Python;
- SQLAlchemy;
- Psycopg;
- Pandas;
- Matplotlib.

O SQLAlchemy será utilizado para gerenciamento das conexões e execução das consultas SQL. O Psycopg atuará como driver de comunicação com o PostgreSQL. O Pandas será empregado na manipulação e comparação dos resultados retornados pelas consultas, enquanto o Matplotlib será utilizado para geração dos gráficos e análises estatísticas.

4.5. Modelo de Linguagem Utilizado

A etapa de reescrita será realizada por meio de um Modelo de Linguagem de Grande Escala capaz de executar tarefas de SQL-to-SQL Query Rewriting.

Os experimentos poderão utilizar modelos como GPT-4o, GPT-5, DeepSeek ou Llama 3, dependendo da disponibilidade de acesso e dos recursos computacionais disponíveis durante a execução do estudo.

A escolha do modelo será baseada na qualidade das reescritas produzidas, na estabilidade dos resultados obtidos e na viabilidade de utilização dentro do ambiente experimental.

4.6. Conjunto de Dados

A avaliação experimental utilizará bases amplamente empregadas na literatura de otimização de consultas e desempenho de bancos de dados.

Entre os conjuntos de dados considerados destacam-se o TPC-H, o TPC-DS e o Join Order Benchmark (JOB), construído a partir da base IMDb. Esses benchmarks contêm consultas analíticas com diferentes níveis de complexidade, envolvendo múltiplas junções, agregações e operações de filtragem.

A utilização desses conjuntos permite comparar os resultados obtidos com trabalhos relacionados e garantir maior reprodutibilidade dos experimentos.

4.7. Cenários de Avaliação

A avaliação será conduzida em três cenários distintos.

No primeiro cenário, as consultas serão executadas sem qualquer modificação, utilizando exclusivamente o otimizador nativo do PostgreSQL.

No segundo cenário, as consultas serão reescritas pelo modelo de linguagem antes da execução, sem aplicação de mecanismos adicionais de validação.

No terceiro cenário, as consultas reescritas serão submetidas ao processo completo de validação proposto neste trabalho, envolvendo verificações sintáticas, semânticas e análise dos planos de execução.

A comparação entre os três cenários permitirá avaliar não apenas o potencial de otimização dos LLMs, mas também a eficácia dos mecanismos de validação na prevenção de regressões de desempenho e erros semânticos.

4.8. Métricas de Avaliação

A análise dos resultados será realizada por meio de métricas objetivas de desempenho.

Serão observados o tempo total de execução, o custo estimado do plano de execução, a quantidade de linhas processadas, a taxa de sucesso das reescritas, o percentual de consultas efetivamente otimizadas e a quantidade de reescritas rejeitadas durante o processo de validação.

Os resultados obtidos serão comparados estatisticamente entre os diferentes cenários experimentais, permitindo verificar a efetividade da abordagem proposta e sua viabilidade para aplicação em ambientes reais. “‘

5. Resultados e Discussão

Essa seção deverá ser escrita na segunda parte do trabalho, conhecida como TCC2, e deverá conter os resultados dos experimentos realizados, discussão comparando os resultados obtidos com outros encontrados em trabalhos similares, além de um parágrafo apontando as limitações da metodologia adotada.



Figura 1. Exemplo de uso de figura

Figura 2. Exemplo com sub-figuras



(a) Sub-figura 1



(b) Sub-figura 2

Tabela 1. Minha tabela

cabeçalho 1	**cabeçalho 2**
texto à esquerda	Existem muitas variações das passagens do Lorem Ipsum disponíveis, mas a maior parte sofreu alterações de alguma forma.

Tabela 2. Tabela com sub-tabelas

cabeçalho 1	cabeçalho 2
Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old.	The standard chunk of Lorem Ipsum used since the 1500s is reproduced below for those interested.

(a) Sub-tabela da esquerda

cabeçalho 1	cabeçalho 2
Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old.	The standard chunk of Lorem Ipsum used since the 1500s is reproduced below for those interested.

(b) Sub-tabela da direita

6. Considerações Finais

Essa seção deverá ser escrita na segunda parte do trabalho, conhecida como TCC2.

Referências

DHARWADA, S. *et al.* LITHE: A Query Rewrite Advisor using LLMs. *In:* INTERNATIONAL CONFERENCE ON EXTENDING DATABASE TECHNOLOGY. **Proceedings of the 29th International Conference on Extending Database Technology (EDBT 2026)**. Tampere, Finland: OpenProceedings.org, 2026. Disponível em: https://openproceedings.org/2026/conf/edbt/paper-93.pdf. Acesso em: 9 jun. 2026.

LI, Z. *et al.* LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. **Proceedings of the VLDB Endowment**, v. 18, n. 1, p. 53–65, 2024. Disponível em: https://www.vldb.org/pvldb/vol18/p53-yuan.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; TATBUL, N. *et al.* Bao: Making Learned Query Optimization Practical. *In:* ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA. **Proceedings of the 2021 International Conference on Management of Data**. Virtual Event, China: ACM, 2021. p. 1275–1288. DOI: 10.1145/3448016.3452838. Disponível em: https://people.csail.mit.edu/hongzi/content/publications/BAO-Sigmod21.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; ZHANG, C. *et al.* Neo: A Learned Query Optimizer. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1705–1718, 2019. Disponível em: https://www.vldb.org/pvldb/vol12/p1705-marcus.pdf. Acesso em: 9 jun. 2026.

SONG, Y. *et al.* **QUITE: A Query Rewrite System Beyond Rules with LLM Agents**. [*S. l.: s. n.*], 2025. arXiv preprint, abs/2506.07675. DOI: 10.48550/arXiv.2506.07675. Disponível em: https://arxiv.org/abs/2506.07675. Acesso em: 9 jun. 2026.

SUN, Z. *et al.* R-Bot: An LLM-based Query Rewrite System. **Proceedings of the VLDB Endowment**, v. 18, n. 12, p. 5031–5044, 2025. Disponível em: https://www.vldb.org/pvldb/vol18/p5031-li.pdf. Acesso em: 9 jun. 2026.

YANG, Z. *et al.* NeuroCard: One Cardinality Estimator for All Tables. **Proceedings of the VLDB Endowment**, v. 14, n. 1, p. 61–73, 2021. Disponível em: https://www.vldb.org/pvldb/vol14/p61-yang.pdf. Acesso em: 9 jun. 2026.

ZHU, R. *et al.* FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. **Proceedings of the VLDB Endowment**, v. 14, n. 9, p. 1489–1502, 2021. Disponível em: https://doi.org/10.14778/3461535.3461539. Acesso em: 9 jun. 2026.